

Print Management System Technical Documentation

System design, API, deployment, maintenance, validation, and onboarding guide

Repository	print-management-system
Document date	May 6, 2026
Primary stack	React, TypeScript, Vite, Express, PostgreSQL, plain SQL migrations, Windows print connector
System status	Full-stack MVP foundation with implemented secure print release path, in-progress Active Directory integration, and deployment hardening still required.

Table of Contents

- 1. Executive Overview
- 2. System Architecture
- 3. Detailed System Design
- 4. API Documentation
- 5. Deployment and Infrastructure
- 6. Maintenance and Extensibility Guide
- 7. Testing and Validation
- 8. Active Directory Integration Plan
- 9. Onboarding Guide
- 10. Future Improvements

1. Executive Overview

1.1 System Purpose

The Print Management System is a university-focused web application being built as an in-house replacement for PaperCut. It supports print job submission, queue management, quota control, printer administration, operational logs, technician alert handling, and secure release of held print jobs.

The repository is no longer only a frontend prototype. It contains a React frontend, a TypeScript and Express backend, PostgreSQL migrations, database-backed authentication for development, API-backed admin and technician workflows, and diagnostic and product-oriented print delivery paths.

1.2 Key Features

Feature	Status	Repository evidence
Role-based sign-in with JWT sessions	Implemented	Backend auth routes, JWT helper, seeded database credentials, frontend sign-in page.
Standard-user print portal	Implemented	Portal dashboard, submit job, history, cancellation, and stored-job actions.
Administrator portal	Implemented	Dashboard, users, groups, printers, queues, logs, and management screens.
Technician portal	Implemented	Dashboard, users, printers, and alerts screens with role-limited backend access.
PostgreSQL-backed job, queue, user, printer, group, log, and alert data	Implemented	SQL migrations under backend/migrations and services under backend/src/services.
Secure print release	Implemented	Implemented as an MVP HP PjL stored/private job flow using POST /api/jobs/:id/store-on-device and the Windows connector /store-on-device endpoint. A generated PIN is stored encrypted in the database and returned to the user for printer-panel release.
Windows connector and fallback diagnostic print paths	Implemented	Connector process supports Windows queue submission, raw PDF upload, and HP PjL stored-job submission. Direct raw socket printing remains a diagnostic path.
Active Directory login and identity sync	In Progress	The schema includes AD object and group fields, the frontend references AD status, and SSO Test 1.py contains a Microsoft identity proof of concept. The Express backend still uses database credentials for authentication.
Physical printer completion telemetry	Planned	The backend records submission and stored-job state, but does not confirm final paper output, panel authentication, jams, toner state, or device completion.
Production cleanup of expired jobs and stored files	Planned	The schema tracks expiry and file deletion timestamps, but no scheduled cleanup service is implemented in the repository.

1.3 Current System Status

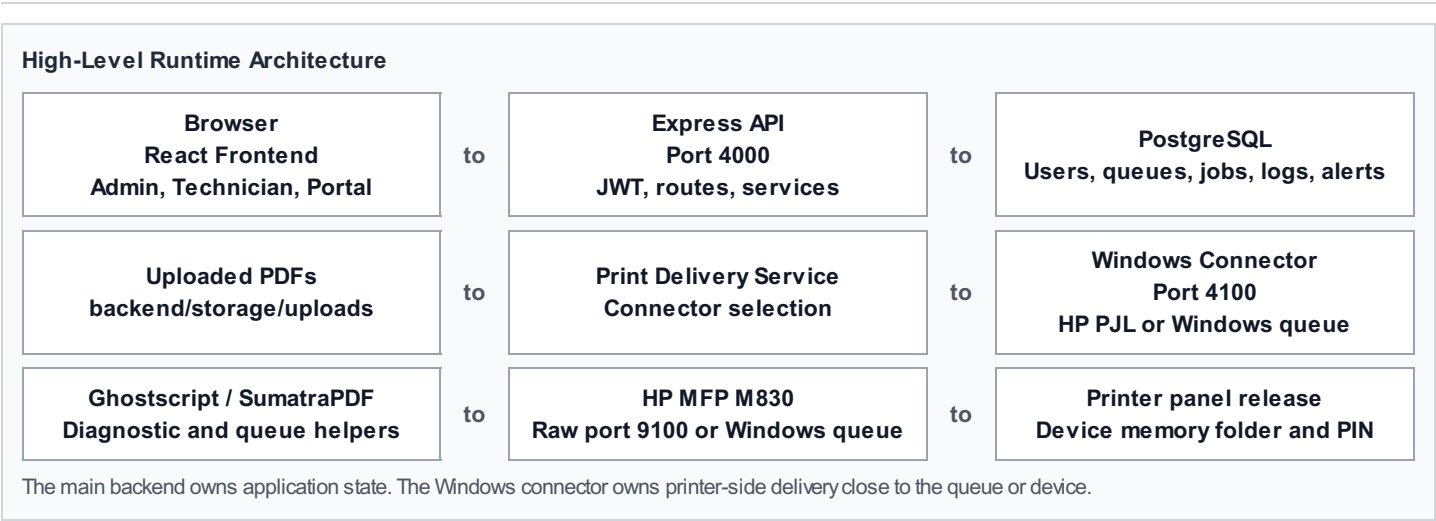
The system is a full-stack MVP foundation. Local development and VM-based pilot operation are supported when PostgreSQL, the backend, the frontend, and the Windows print connector are configured correctly. The secure release flow is implemented for HP PjL stored/private jobs with PIN-to-print semantics. Active Directory integration is in progress and should be treated as a planned authentication replacement, not as the current production login mechanism.

1.4 Deployment Readiness

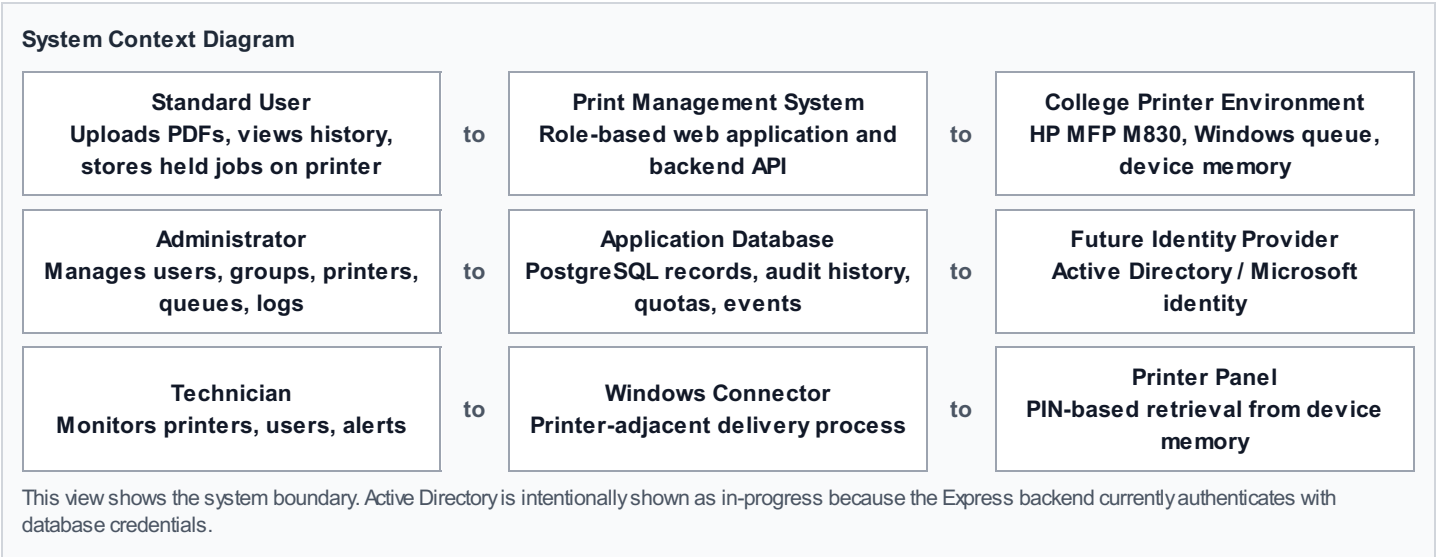
Area	Readiness	Notes
Local developer setup	Ready	npm scripts, Docker Compose for PostgreSQL, migrations, and Vite frontend scripts are present.
College VM pilot	Ready with configuration	Requires Node.js, PostgreSQL connectivity, backend .env values, firewall rules for ports 4000 and 4100, and a running Windows connector close to the printer queue.
Production deployment	Not fully production-hardened	Requires AD integration, secret management, connector authentication, retention cleanup, operational monitoring, and completion telemetry.

2. System Architecture (High-Level Design)

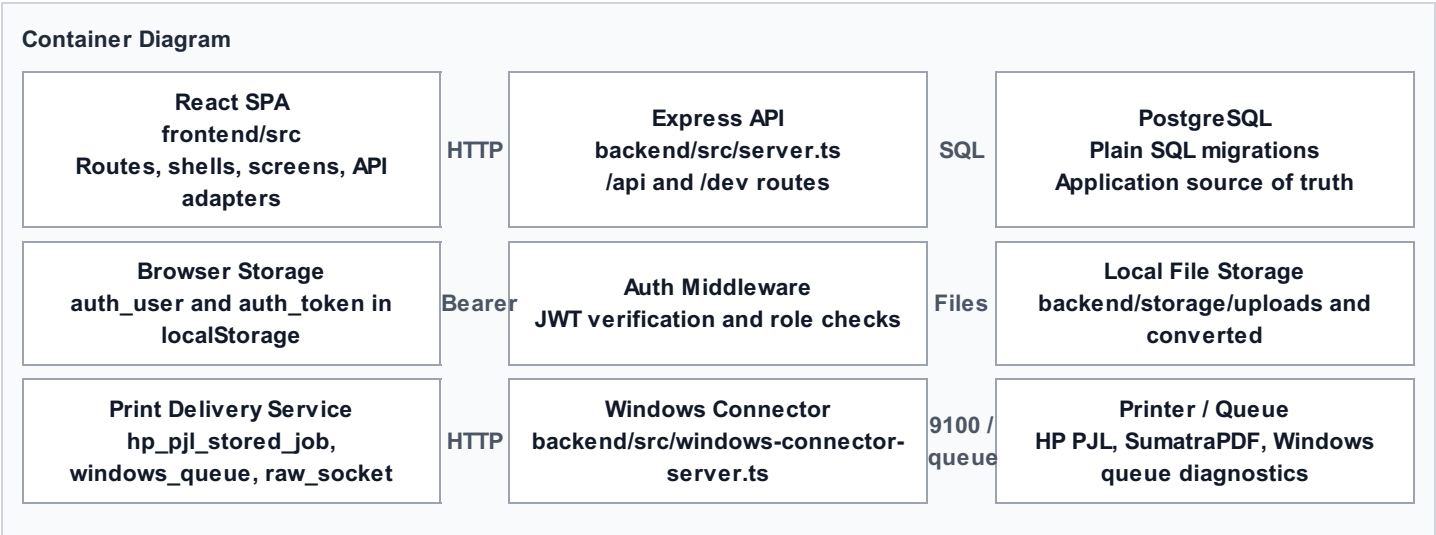
2.1 Overall Architecture Diagram



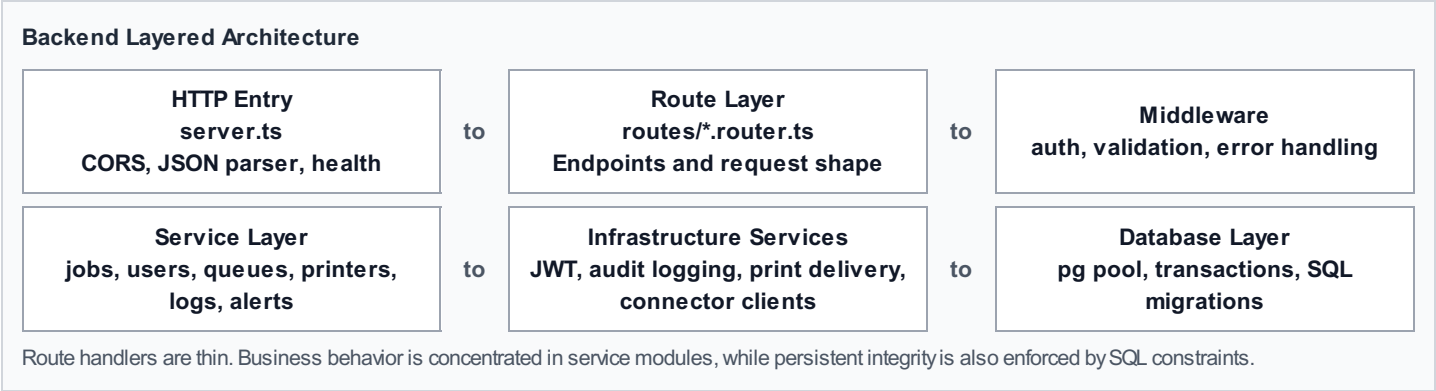
2.1.1 System Context View



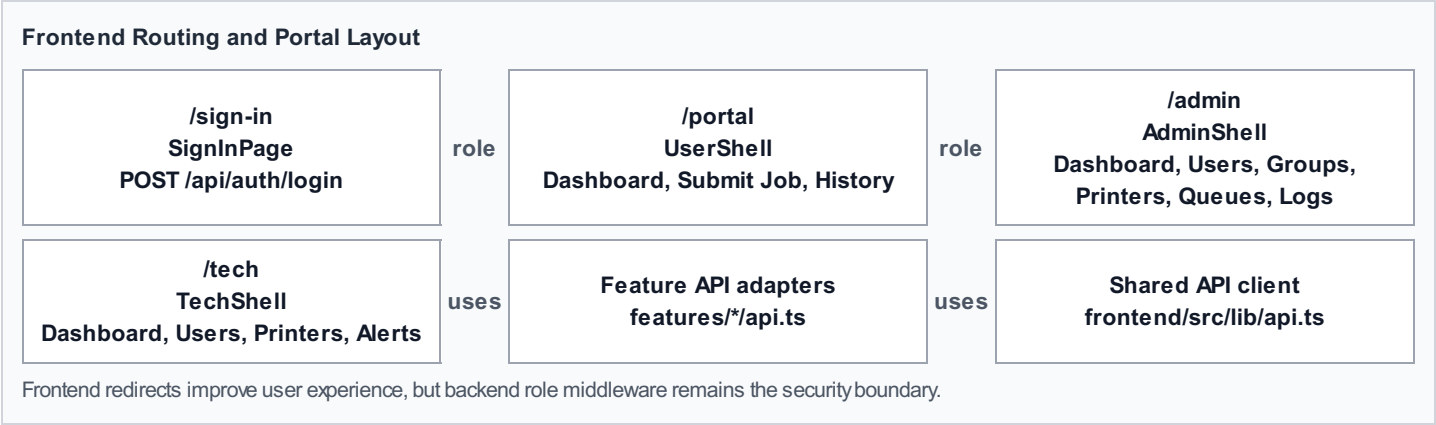
2.1.2 Container View



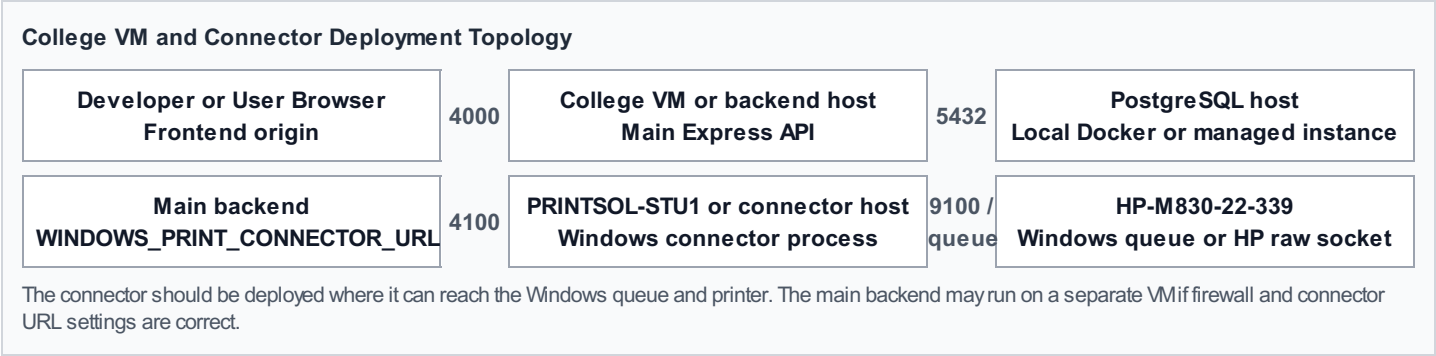
2.1.3 Backend Layer Diagram



2.1.4 Frontend Route and Portal Diagram



2.1.5 Deployment Topology Diagram



2.2 Frontend and Backend Structure

Layer	Technology	Primary responsibilities
Frontend	React, TypeScript, Vite, React Router, Tailwind CSS, Radix primitives, lucide-react icons	Role-specific portals, API calls, local session storage, forms, tables, dashboards, and operational UI.
Main backend	TypeScript, Express, pg, zod, multer, pdf-lib	Authentication, authorization, request validation, job submission, PDF upload, quota checks, queue resolution, database writes, logs, alerts, and print delivery orchestration.
Database	PostgreSQL with plain SQL migrations	Stores identity records, roles, groups, queues, printers, jobs, files metadata, events, logs, alerts, and audit data.
Print connector	Separate Express process in backend/src/windows-connector-server.ts	Receives PDF payloads from the main backend and submits them to HP PJI stored-job flow or Windows queue diagnostics.

2.3 Multi-Portal System Design

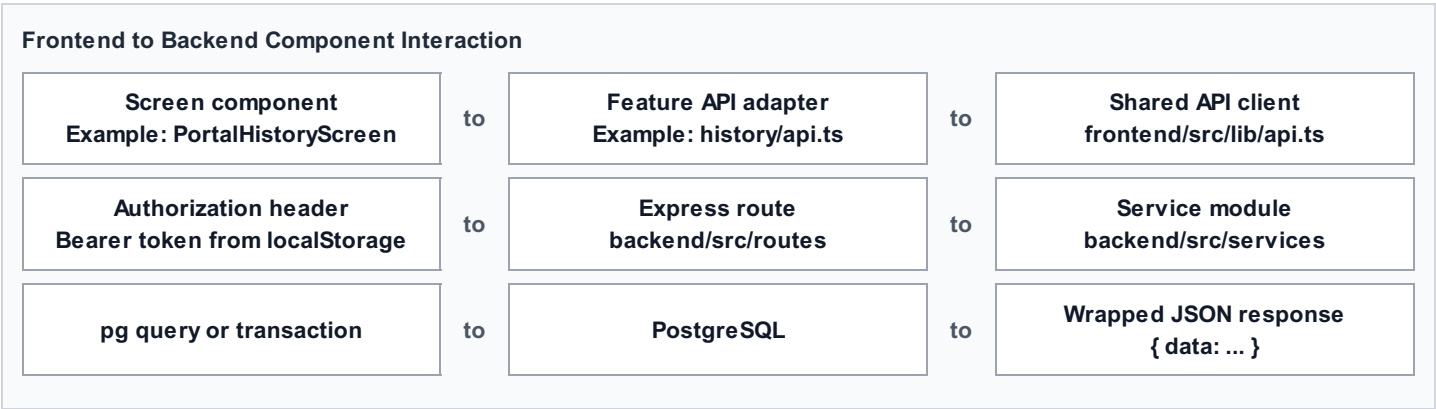
The frontend router exposes three role-oriented areas: /portal for standard users and faculty-style users, /admin for administrators, and /tech

for technicians. The sign-in page maps backend roles to frontend role labels and redirects users to the appropriate portal after authentication.

Portal	Implemented routes	Main capabilities
User portal	/portal/dashboard, /portal/submit-job, /portal/history	View eligible queues and quota context, submit PDF jobs, cancel held jobs, store jobs on the printer, and reveal stored-job PINs.
Admin portal	/admin/dashboard, /admin/users, /admin/groups, /admin/printers, /admin/queues, /admin/logs, /admin/about	Manage system records, view dashboards, review logs, and administer users, groups, printers, and queues.
Technician portal	/tech/dashboard, /tech/users, /tech/printers, /tech/alerts	Monitor printer health, view users and printers, and acknowledge operational alerts. Backend role checks limit technician actions.

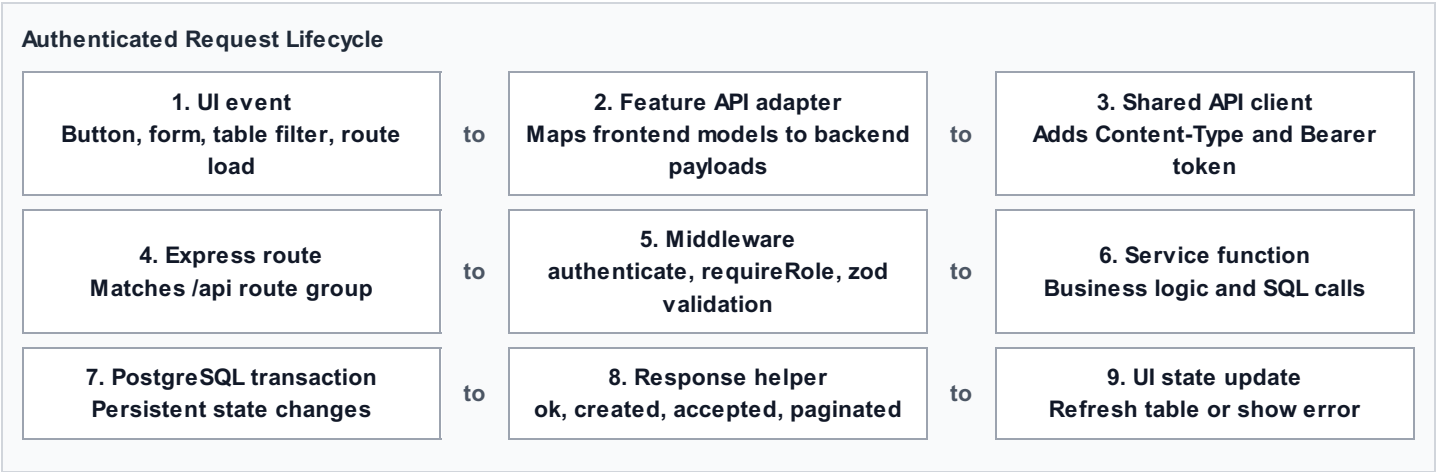
The frontend chooses the portal after login, but the backend is the authority for permissions. API route middleware enforces Bearer token authentication and role checks for administrator and technician-only route groups.

2.4 Component Interaction

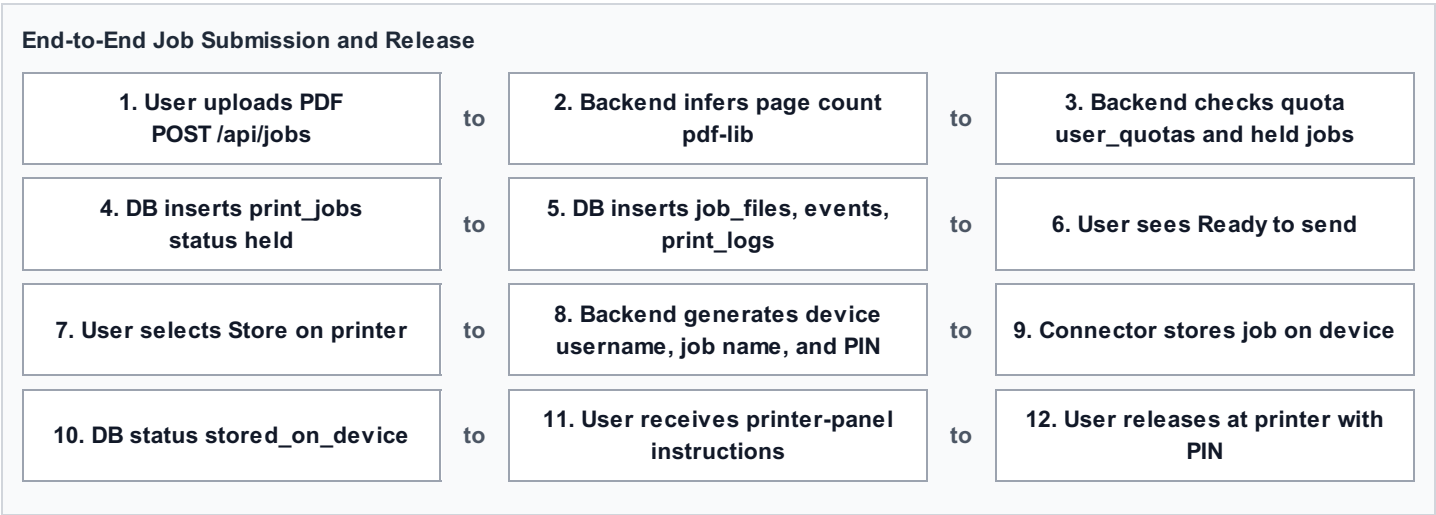


2.5 Data Flow

API calls flow from feature-specific frontend adapters to the shared API client. The shared client attaches the JWT from localStorage. The backend authenticates the token, validates query strings or bodies with zod, delegates to a service module, and returns either a standard response wrapper or a paginated result. Database changes are handled through direct SQL queries and explicit transactions.



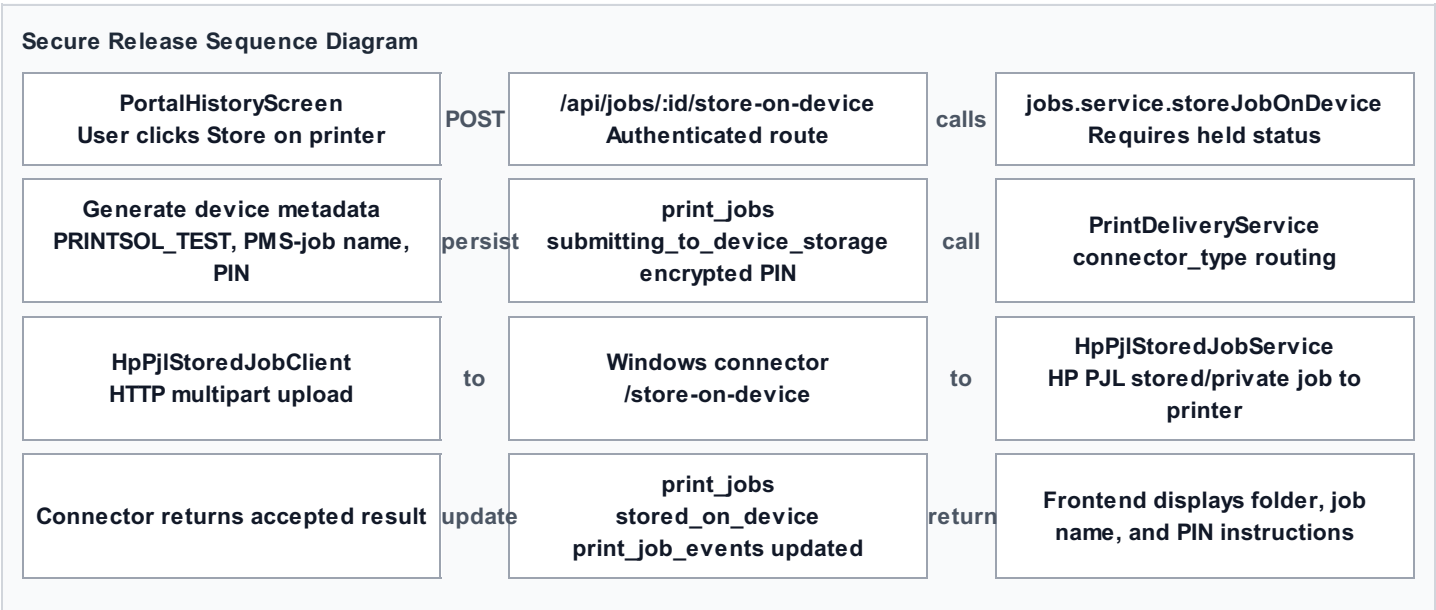
2.6 Print Workflow (End-to-End)



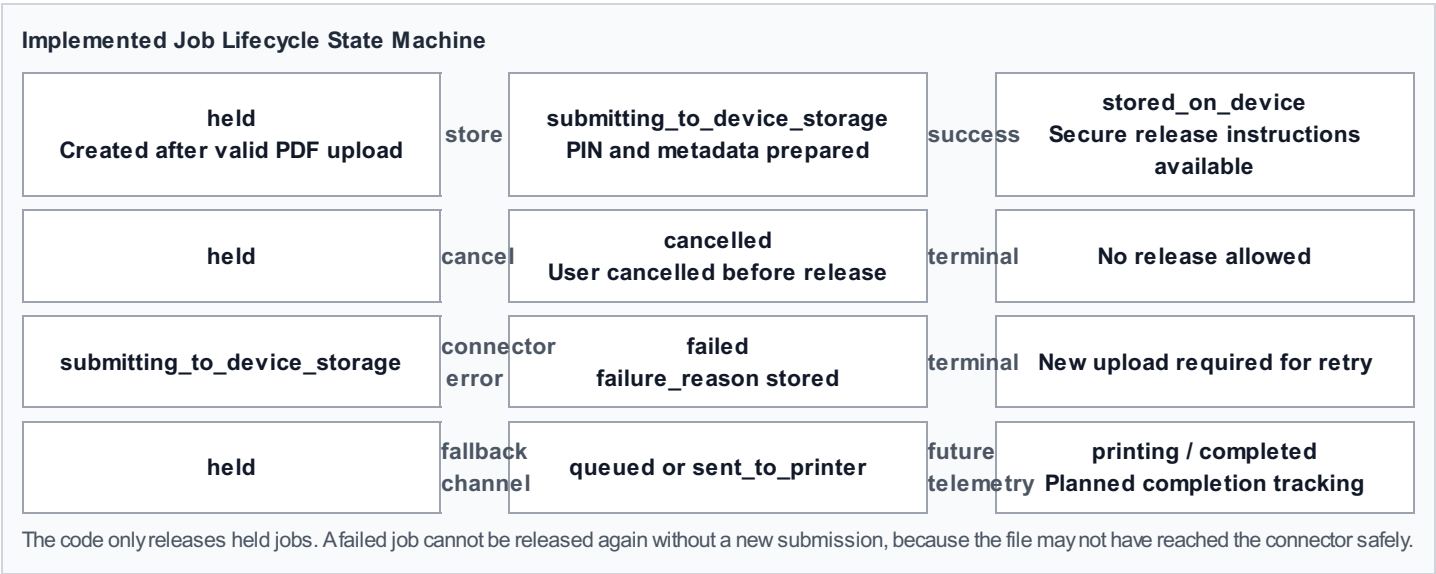
2.7 Secure Print Release Workflow

Secure print release is implemented as an MVP HP PjL stored/private job workflow. The backend requires the job status to be held before release. It generates a PIN, encrypts the PIN with JOB_PIN_ENCRYPTION_KEY, stores device metadata on the job record, and calls the Windows connector. The connector sends a stored/private job to the printer so the user can retrieve it from device memory using the folder name, job name, and PIN.

The implemented secure release flow does not yet provide final physical completion telemetry. It confirms backend state changes and connector submission, not final paper output or printer-panel authentication success.



2.7.1 Job Status State Machine



2.7.2 Connector Selection Decision Tree



2.8 Authentication Design (Current and Active Directory Plan)

Area	Status	Design
Current authentication	Implemented	Database-backed login against user_credentials. Passwords are hashed with bcrypt. JWTs are HMAC SHA-256 signed and expire after 24 hours.
Current authorization	Implemented	Express middleware validates Bearer tokens and requires admin or technician roles on protected route groups.
Active Directory integration	In Progress	The schema includes ad_object_id, ad_groups, and user_ad_group_memberships. A Microsoft identity proof of concept exists in SSO Test 1.py. Integration into the Express authentication flow remains to be implemented.

2.9 Deployment on College VM

The repository is designed for a college VM or lab environment where the main backend can run separately from the Windows print connector. The connector should run close to the Windows print queue or printer network path. Repository documentation identifies PRINTSOL-STU1 and queue HP-M830-22-339 as the intended Windows-side target, with HP MFP M830 raw printer delivery on port 9100.

Service	Default port	Purpose
Frontend development server	5173	Vite local frontend.
Main backend	4000	Application API, health check, job and management routes.
Windows print connector	4100	Printer-side connector for Windows queue submission and HP PjL stored jobs.
PostgreSQL	5432	Application database.
HP printer raw socket	9100	Raw printer protocol used by HP PjL stored-job and diagnostic paths.

3. Detailed System Design (Low-Level)

3.1 Folder Structure Explanation

Path	Purpose
frontend/src/app	Application router and role-specific shell layouts.
frontend/src/features	Feature modules for admin, portal, and technician areas. Each feature includes screens and API adapters.
frontend/src/components	Reusable and domain-specific UI components for dashboards, tables, dialogs, queues, printers, users, groups, logs, and alerts.
frontend/src/lib	Shared API client, auth storage, formatting, chart helpers, status helpers, and utilities.
frontend/src/types	TypeScript types for auth, admin, portal, and technician domains.
frontend/src/mocks	Mock stores retained for earlier or fallback UI data modeling.
backend/src/routes	Express routers for API groups and diagnostic print routes.
backend/src/services	Business logic, database operations, print delivery, connector clients, PDF conversion, and audit logging.
backend/src/middleware	Authentication, role authorization, validation, and error handling middleware.
backend/src/db	PostgreSQL pool and migration runner.
backend/migrations	Plain SQL migrations and development seed data. This is the implementation source of truth for schema changes.
backend/storage/uploads	Local uploaded PDF storage. The database stores metadata and paths, not file bytes.
backend/storage/converted	Converted PostScript output for raw socket diagnostic printing.

3.2 Key Modules and Responsibilities

Module	Responsibility
backend/src/server.ts	Configures CORS, JSON parsing, health route, /api routes, /dev routes, and error middleware.
backend/src/config.ts	Loads and validates environment variables with zod and resolves storage paths.
backend/src/routes/index.ts	Mounts the main API routers under /api.
backend/src/services/jobs.service.ts	Handles job submission, PDF page count inference, quota checks, secure release, job cancellation, device PIN retrieval, event logging, and print log updates.
backend/src/services/print-delivery-service.ts	Selects delivery channel based on printer connector_type: hp_pjl_stored_job, windows_queue, or raw_socket.
backend/src/windows-connector-server.ts	Runs the connector process for printer-adjacent delivery operations.
frontend/src/lib/api.ts	Defines the shared fetch wrapper, backend base URL, JSON handling, FormData handling, and Bearer token injection.
frontend/src/app/router.tsx	Defines portal, admin, technician, sign-in, and not-found routes.

3.3 Backend Services and Controllers

Router	Service	Scope
auth.router.ts	auth.service.ts	Login, development login alias, logout response, and current user profile.
portal.router.ts	auth.service.ts, queues.service.ts, jobs.service.ts	Portal profile, eligible queues, and dashboard snapshot.
jobs.router.ts	jobs.service.ts	Job listing, upload, history, events, secure release, PIN reveal, release alias, and cancellation.
users.router.ts	users.service.ts	User listing, create, update, suspend, reactivate, quota update, delete, and group list.
printers.router.ts	printers.service.ts	Printer listing, create, read, update, archive, and error history.
queues.router.ts	queues.service.ts	Queue listing, eligible queues, create, detail, update, and archive.
groups.router.ts	groups.service.ts	AD-style group management and group member listing.
alerts.router.ts	alerts.service.ts	Operational alert listing, detail, and acknowledgement.
logs.router.ts	logs.service.ts	Log overview and operational event listing.
dashboard.router.ts	dashboard.service.ts	Admin and technician dashboard snapshots, print logs, and print activity chart data.

3.3.1 Backend Service Detail Map

Service file	Key functions or classes	Detailed responsibility
auth.service.ts	login, getMe, logAuthAttempt	Looks up users by email or username, verifies script password hashes, rejects suspended accounts, signs JWTs, and records authentication attempts.
users.service.ts	listUsers, createUser, updateUser, suspendUser, reactivateUser, deleteUser	Handles user CRUD, role assignment, quota updates, AD-style group membership, technician restrictions, and audit logging for user management changes.
groups.service.ts	listGroups, getGroupByPublicId, listGroupUsers, createGroup, updateGroup, deleteGroup	Manages AD-style groups, quota policy metadata, default group flags, membership reporting, and cleanup of dependent queue access rules.
queues.service.ts	listQueues, listEligibleQueuesForUser, resolveDefaultQueueForUser, createQueue, updateQueue, deleteQueue	Builds queue listings, resolves portal eligibility from roles and groups, manages printer assignments, access rules, pricing, and queue audit logs.
printers.service.ts	listPrinters, getPrinterById, createPrinter, updatePrinter, deletePrinter, getPrinterErrors	Maintains printer metadata, connector settings, operational status, toner level, queue association summaries, and archive behavior.
jobs.service.ts	submitJob, listJobs, storeJobOnDevice, releaseJob, getDevicePin, cancelJob, listJobEvents	Owns the print job lifecycle, quota checks, PDF page inference, file metadata, secure release, PIN encryption, connector error handling, and job event history.
print-delivery-service.ts	PrintDeliveryService.deliverPdf	Selects the delivery implementation based on printer connector_type and normalizes delivery results into job status updates.
hp-pjl-stored-job-client.ts	storeUploadedPdf	Builds multipart requests from uploaded PDFs and device storage metadata, then calls the connector /store-on-device endpoint.
hp-pjl-stored-job-service.ts	storePdfOnDevice, sendDiagnosticStoredJob	Connector-side implementation that wraps PDFs for HP PJI stored/private job behavior and sends bytes to the printer target.
windows-queue-print-client.ts	printUploadedPdf	Main-backend client for connector-based Windows queue diagnostics.
windows-print-command.ts	printPdf	Connector-side command execution for SumatraPDF, PowerShell, or custom command print modes.
audit-log.service.ts	recordAuditLog, recordAuditLogWithClient	Centralized audit insertion with optional transaction client support.

3.4 Frontend Components and Pages

The frontend uses a feature-oriented structure. Screens call feature API adapters, adapters call the shared API client, and components render tables, forms, dialogs, status badges, filters, dashboards, and detail panels. The admin and technician sections share many concepts but map them into role-specific screens.

Area	Representative files	Responsibilities
Portal	features/portal/dashboard, features/portal/submit-job, features/portal/history	Queue selection context, PDF upload, job history, cancellation, secure printer storage, and PIN reveal.
Admin	features/admin/dashboard, users, groups, printers, queues, system-logs	Operational dashboards and administrative CRUD workflows.
Technician	features/technician/dashboard, users, printers, alerts	Support-oriented monitoring and limited user/printer/alert operations.
Shared UI	components/ui and components/composite	Buttons, dialogs, tables, inputs, tabs, sheets, badges, filters, page headers, and action rails.

3.4.1 Frontend Data Flow by Portal

Frontend feature	API adapter	Backend routes used	Local mapping responsibility
Portal dashboard	features/portal/dashboard/api.ts	/portal/dashboard and /jobs/:id/cancel	Maps backend profile, queues, and jobs into portal dashboard cards and active job lists.
Portal submit job	features/portal/submit-job/api.ts	/portal/queues and /jobs	Builds FormData for PDF uploads and maps backend job statuses to user-facing labels such as Ready to send and Stored on printer.
Portal history	features/portal/history/api.ts	/jobs/my, /jobs/:id/cancel, /jobs/:id/store-on-device, /jobs/:id/device-pin	Refreshes history rows, stores device release data in component state, and displays printer-panel instructions.
Admin dashboard	features/admin/dashboard/api.ts	/dashboard, /dashboard/print-logs, /dashboard/print-activity	Maps backend dashboard aggregates and chart data into visual dashboard panels.
Admin management pages	features/admin/users/api.ts, groups/api.ts, printers/api.ts, queues/api.ts	/users, /groups, /printers, /queues	Converts frontend form naming to backend input naming and maps backend snake_case fields into UI-facing models.
Technician pages	features/technician/*api.ts	/dashboard/technician, /users, /printers, /alerts	Reuses admin API shapes where appropriate while presenting technician-specific workflows and restrictions.

3.4.2 Frontend State and Session Model

The frontend stores the authenticated user object under `auth_user` and the JWT under `auth_token` in `localStorage`. The sign-in page maps backend roles into frontend labels and redirects to `/admin/dashboard`, `/tech/dashboard`, or `/portal/dashboard`. The shared API client reads `auth_token` on every request and attaches it as a Bearer token.

Feature screens generally keep transient UI state locally with React `useState`. Examples include portal history job rows, working job identifiers, device release PIN display state, table filters, and form drafts. Persistent system state remains in PostgreSQL through backend APIs.

3.5 API Layer Design

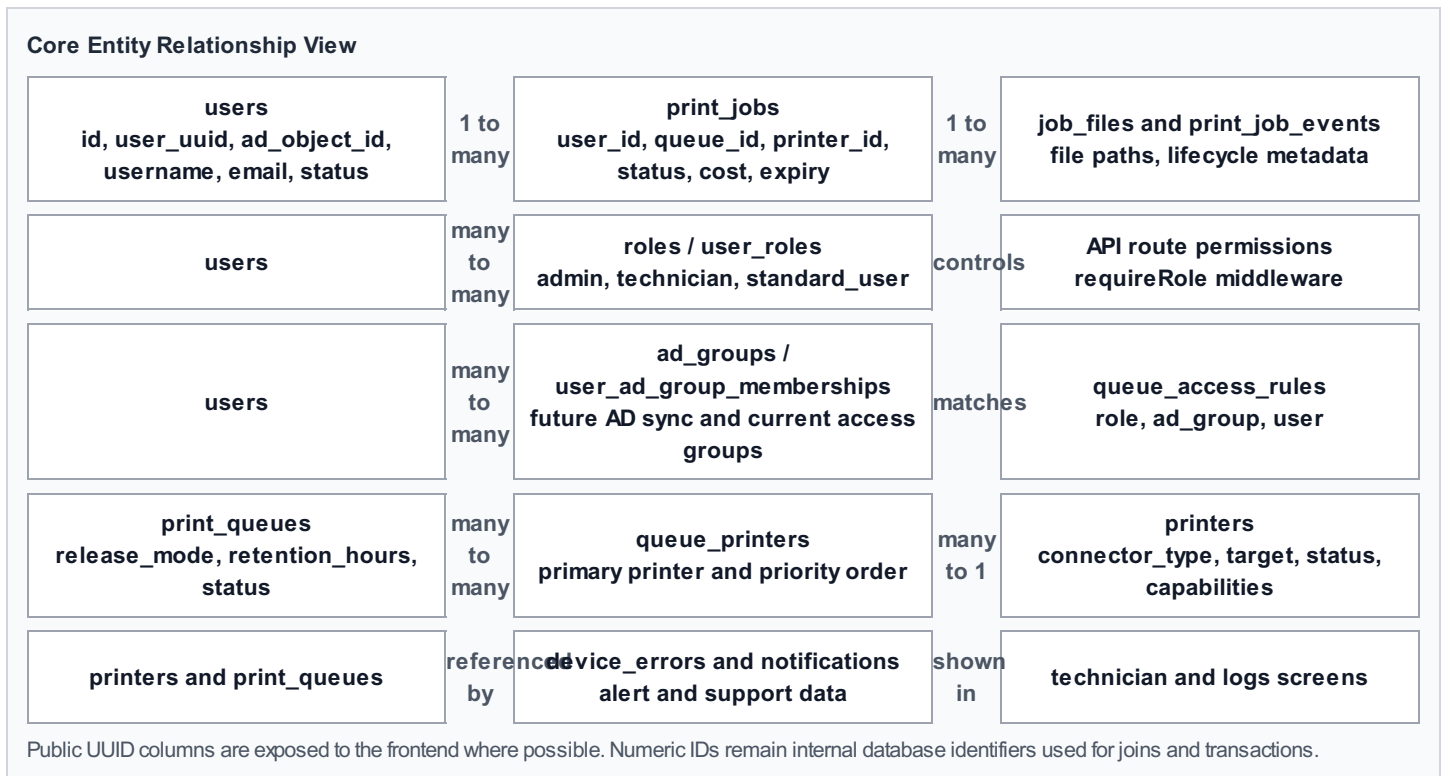
The frontend API client reads `VITE_BACKEND_URL` with a default of `http://localhost:4000`. It prefixes application requests with `/api`, serializes normal request bodies as JSON, preserves FormData for PDF upload, and attaches `Authorization: Bearer <token>` when a token is stored.

Backend route handlers use response helper functions. Most successful responses use `{ data: ... }`. Created resources return HTTP 201. Accepted asynchronous or connector-style actions return HTTP 202. Paginated endpoints return `data`, `total`, `page`, `limit`, and `totalPages`.

3.6 Database and Schema

Table	Purpose
users	Application users, AD object reference, university ID, status flags, and profile fields.
roles and user_roles	Application role assignment for admin, technician, and standard_user.
user_credentials	Temporary development password hashes for DB-backed login.
technician_privileges	Technician capability flags.
user_quotas	Allocated and used page quota per quota period.
ad_groups and user_ad_group_memberships	AD-style group metadata and membership used for queue access and future AD sync.
printers	Physical or logical printer metadata, connector type, connector target, status, toner, serial number, and notes.
print_queues	Logical queues, release mode, default status, retention policy, and queue state.
queue_printers	Assignment of printers to queues, including primary and priority flags.
queue_access_rules	Queue eligibility by role, AD group, or specific user.
pricing_rules	Cost rules by queue, printer, paper type, and color mode.
print_jobs	Job lifecycle state, user, queue, printer, page count, copy count, cost, expiry, release metadata, and device storage metadata.
job_files	File metadata and filesystem paths for original, converted, or spool files.
print_job_events	Detailed event history for each job.
print_logs	Operational print log records for dashboard and log views.
device_errors	Printer and queue error records used by technician alerts.
notifications	Notification records related to device errors and user/role targets.
auth_logs	Successful and failed authentication attempts.
audit_logs	Administrative and operational change history with before and after state.

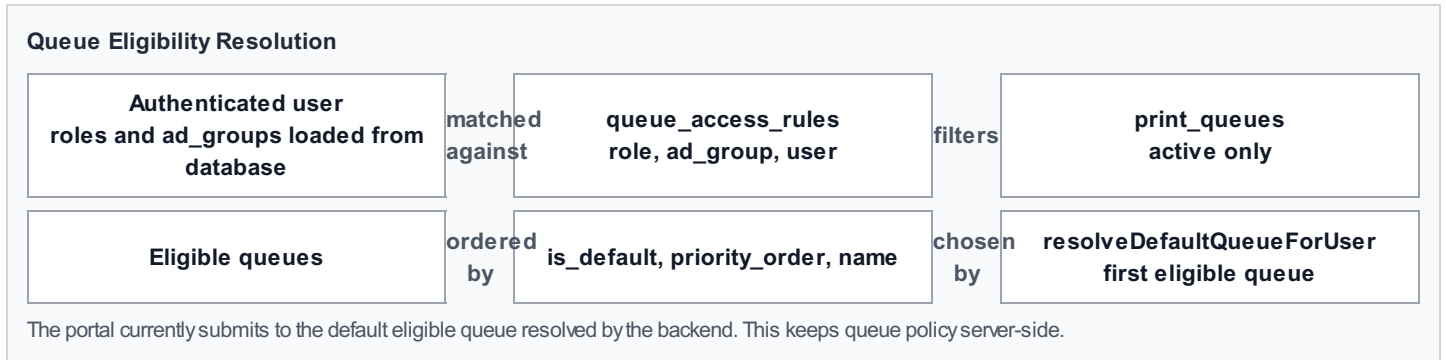
3.6.1 Simplified Database Relationship Diagram



3.6.2 Job and File Storage Model

Record	Important fields	Design reason
print_jobs	status, user_id, queue_id, printer_id, page_count, copy_count, expires_at, device_storage_username, device_storage_job_name, device_storage_pin_secret	Central lifecycle record. Stores release state, quota data, and encrypted PIN metadata without storing the PDF bytes.
job_files	print_job_id, file_role, original_file_name, stored_file_path, mime_type, file_size, file_hash, deleted_at	Keeps physical file locations and metadata outside the job table. Supports original uploads and converted files.
print_job_events	print_job_id, actor_user_id, event_type, event_source, message, metadata	Provides chronological debugging and auditability for job lifecycle transitions.
print_logs	print_job_id, user_id, printer_id, device_name, printed_at, pages, cost, status	Feeds dashboard and operational log views. It is updated as jobs move through held, queued, stored_on_device, printing, failed, and cancelled states.

3.6.3 Queue Access Model



3.7 Core Logic: Print Job Handling, Queueing, and Release

1. Submit job: The backend accepts one PDF file up to 25 MB through multipart field file and validates copyCount.
2. Page count: The backend reads the PDF with pdf-lib and stores page_count_source as pdf_inferred.
3. Queue resolution: The backend selects the first eligible queue returned by listEligibleQueuesForUser, ordered by default flag, priority, and name.
4. Quota enforcement: The backend subtracts used pages and reserved held pages from allocated quota before accepting a job.
5. Database write: The backend inserts print_jobs, job_files, print_job_events, and print_logs records in a transaction.
6. Store on device: The backend requires status held, generates a device folder, job name, and PIN, encrypts the PIN, and calls the configured connector.
7. State update: Successful HP PJI delivery sets status stored_on_device. Windows queue fallback sets queued. Raw socket delivery sets sent_to_printer.
8. Error path: Connector failures mark the job failed, record failure_reason, insert a failed event, and update print_logs.

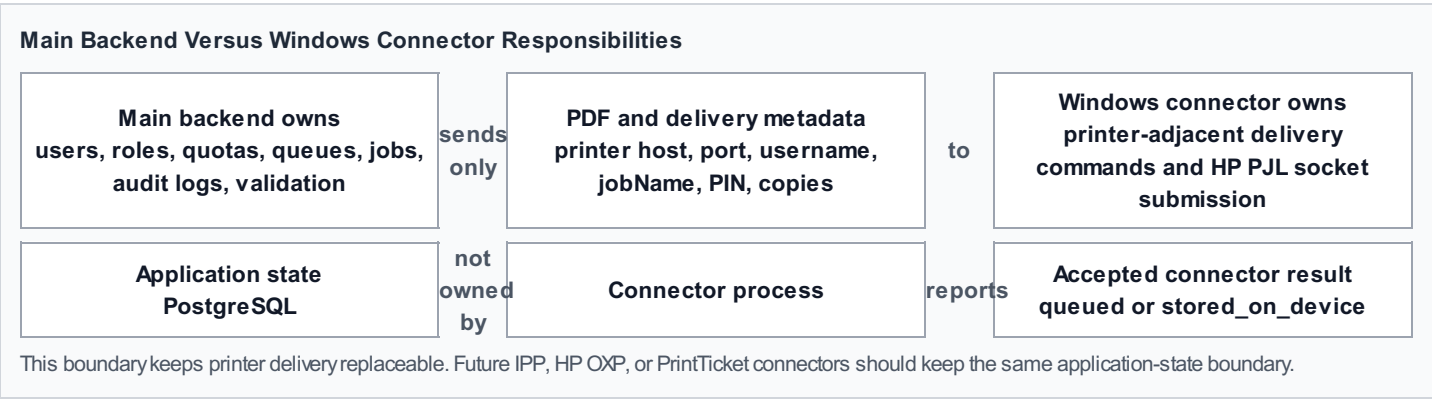
3.7.1 submitJob Internal Steps

Step	Implementation detail	Tables affected
Resolve queue	resolveDefaultQueueForUser loads eligible queues using role, AD group, and user access rules, then picks the highest priority queue.	users, user_roles, ad_groups, user_ad_group_memberships, queue_access_rules, print_queues
Infer pages	inferPdfPageCount reads the uploaded file with pdf-lib and rejects PDFs that cannot be parsed or have no pages.	None before acceptance
Check quota	assertQuotaAvailable compares allocated pages against used pages and reserved pages from active held/submitting/blocked jobs.	user_quotas, print_jobs
Estimate cost	estimateCost reads active pricing rules for the resolved queue, standard paper, black-and-white mode, and total pages.	pricing_rules
Persist job	A transaction inserts print_jobs with status held, job_files metadata, a submitted print_job_events record, and a held print_logs record.	print_jobs, job_files, print_job_events, print_logs

3.7.2 storeJobOnDevice Internal Steps

Step	Implementation detail	Failure behavior
Load release candidate	getReleaseCandidate joins the job, user, original file, queue-printer assignment, and online printer. It requires status held by default.	Returns 404 if no releasable job exists, 403 for standard-user ownership mismatch, or 422 if the job is not held.
Prepare PIN metadata	The backend generates a PIN, builds a sanitized device username and job name, encrypts the PIN, and stores submitting_to_device_storage state.	If this step fails, the job is not sent to the connector.
Deliver through connector	PrintDeliveryService routes by connector_type. The MVP secure path uses hp_pjl_stored_job and calls the Windows connector /store-on-device endpoint.	Connector errors are caught and stored as failure_reason.
Finalize status	Successful HP PJI stored-job delivery sets status stored_on_device, records released_at, stores connector job ID when returned, increments used quota, and updates print_logs.	If a post-send update fails after sentToDeviceStorage is true, the code attempts to recover to stored_on_device to avoid losing the user's PIN path.
Return release info	The response includes the updated job and deviceRelease object with username, jobName, PIN, and instructions.	If delivery did not use the HP stored-job channel, deviceRelease is null.

3.7.3 Print Connector Boundaries



- Active Directory production secrets, VM credentials, printer credentials, and connector tokens must not be committed.

4. API Documentation

4.1 Common API Conventions

Convention	Description
Base URL	Main backend defaults to <code>http://localhost:4000</code> . Frontend application requests use the <code>/api</code> prefix.
Authentication	Protected endpoints require <code>Authorization: Bearer <jwt></code> . The JWT payload includes <code>sub</code> , <code>role</code> , <code>roles</code> , <code>iat</code> , and <code>exp</code> .
Success wrapper	Most successful JSON responses use <code>{ "data": ... }</code> . Paginated endpoints return <code>{ "data": [...], "total": n, "page": n, "limit": n, "totalPages": n }</code> .
Error wrapper	Application errors use <code>{ "error": { "code": "VALIDATION_ERROR", "message": "..." } }</code> .
File upload	PDF upload endpoints require <code>multipart/form-data</code> with field name <code>file</code> . Job submission accepts <code>copyCount</code> as an additional form field.

4.2 Health and Authentication Endpoints

Endpoint name	Method	Route	Description	Request parameters and body example	Response example	Errors and auth
Backend health	GET	<code>/health</code>	Checks whether the main backend process is running.	No parameters. No body.	<code>{ "ok": true }</code>	Public. Unexpected server failures return 500.
Login	POST	<code>/api/auth/login</code>	Authenticates a user by email or username and password.	<code>{ "credential": "student@university.edu", "password": "123456" }</code>	<code>{ "data": { "token": "jwt", "user": { "id": "uuid", "role": "standard_user" } }</code>	Public. 401 for invalid credentials or suspended account. 422 for invalid body.
Development login	POST	<code>/api/auth/dev-login</code>	Alias for the same login implementation, retained for development usage.	<code>{ "credential": "admin@university.edu", "password": "123456" }</code>	<code>{ "data": { "token": "jwt", "user": { "id": "uuid", "role": "admin" } }</code>	Public. Same errors as login.
Logout	POST	<code>/api/auth/logout</code>	Returns a logout acknowledgement. Client clears local storage.	No required body.	<code>{ "data": { "message": "Logged out successfully" } }</code>	Requires Bearer token. 401 for missing or invalid token.
Current user	GET	<code>/api/auth/me</code>	Returns the current authenticated user's profile, roles, groups, quota, and job counts.	No parameters. No body.	<code>{ "data": { "id": "uuid", "email": "student@university.edu", "role": "standard_user" } }</code>	Requires Bearer token. 401 for missing or invalid token.

4.3 Portal Endpoints

Endpoint name	Method	Route	Description	Request parameters and body example	Response example	Errors and auth
Portal profile	GET	<code>/api/portal/profile</code>	Returns the authenticated portal user's profile.	No parameters.	<code>{ "data": { "id": "uuid", "display_name": "Student User", "quota_total": 500 } }</code>	Requires Bearer token. 401 for invalid token.
Eligible queues	GET	<code>/api/portal/queues</code>	Lists active queues the authenticated user can access through role, AD group, or user rules.	No parameters.	<code>{ "data": [{ "id": "queue-uuid", "name": "Student Default Queue", "release_mode": "secure_release" }] }</code>	Requires Bearer token. 401 for invalid token.
Portal dashboard	GET	<code>/api/portal/dashboard</code>	Returns profile, eligible queues, and recent user jobs in one payload.	No parameters.	<code>{ "data": { "profile": {}, "queues": [], "jobs": [] } }</code>	Requires Bearer token. 401 for invalid token.

4.4 Job Endpoints

Endpoint name	Method	Route	Description	Request parameters and body example	Response example	Errors and auth
List jobs	GET	/api/jobs	Lists jobs for administrators and technicians with optional filters.	Query: userId, queueId, status, page, limit.	{ "data": [{ "id": "job-uuid", "status": "held" }], "total": 1, "page": 1, "limit": 20, "totalPages": 1 }	Requires admin or technician. 401, 403, 422.
List my jobs	GET	/api/jobs/my	Lists jobs owned by the authenticated user.	Query: queueId, status, page, limit.	{ "data": [{ "id": "job-uuid", "file_name": "document.pdf", "status": "held" }], "total": 1 }	Requires Bearer token. 401, 422.
Submit job	POST	/api/jobs	Uploads a PDF, infers page count, checks quota, stores file metadata, and creates a held job.	multipart/form-data: file=document.pdf, copyCount=1.	{ "data": { "id": "job-uuid", "status": "held", "page_count": 1, "copy_count": 1 } }	Requires Bearer token. 400 missing file. 422 invalid copy count, quota failure, or PDF parse failure.
Get job	GET	/api/jobs/:id	Returns a single job by UUID or internal ID.	Path: id.	{ "data": { "id": "job-uuid", "queue_name": "Student Default Queue", "status": "held" } }	Requires Bearer token. Standard users can only read their own jobs. 401, 403, 404.
List job events	GET	/api/jobs/:id/events	Returns lifecycle events for a job.	Path: id.	{ "data": [{ "event_type": "submitted", "message": "Job uploaded through portal and held for release." }] }	Requires Bearer token. 401, 403, 404.
Reveal device PIN	GET	/api/jobs/:id/device-pin	Returns printer-panel release instructions and PIN for a stored_on_device job.	Path: id.	{ "data": { "username": "PRINTSOL_TEST", "jobName": "PMS-...", "pin": "1234" } }	Requires Bearer token. 422 if job is not stored_on_device or expired. 401, 403, 404.
Store on device	POST	/api/jobs/:id/store-on-device	Sends a held job to the printer connector as a secure stored/private job and returns release details.	Path: id. No body.	{ "data": { "id": "job-uuid", "status": "stored_on_device", "deviceRelease": { "username": "PRINTSOL_TEST", "pin": "1234" } } }	Requires Bearer token. 422 if status is not held. 500 if connector fetch fails. 401, 403, 404.
Release job	POST	/api/jobs/:id/release	Alias that calls the same store-on-device implementation and returns accepted job data.	Path: id. No body.	{ "data": { "id": "job-uuid", "status": "stored_on_device" } }	Requires Bearer token. Same lifecycle and connector errors as store-on-device.
Cancel job	POST	/api/jobs/:id/cancel	Cancels a held job before release.	Path: id. No body.	{ "data": { "id": "job-uuid", "status": "cancelled" } }	Requires Bearer token. 422 if status is not held. 401, 403, 404.

4.5 Dashboard, Logs, and Alert Endpoints

Endpoint name	Method	Route	Description	Request parameters and body example	Response example	Errors and auth
Admin dashboard	GET	/api/dashboard	Returns system-wide dashboard metrics.	No parameters.	{ "data": { "summary": {}, "printerStatus": [], "recentPrintLogs": [] } }	Requires admin or technician. 401, 403.
Technician dashboard	GET	/api/dashboard/technician	Returns technician-focused printer health and alert data.	No parameters.	{ "data": { "stats": {}, "printer_health": [], "active_alerts": [] } }	Requires admin or technician. 401, 403.
Recent print logs	GET	/api/dashboard/print-logs	Lists recent print log rows for dashboards.	Query: search, status, device, page, limit.	{ "data": { "data": [], "total": 0, "page": 1, "limit": 20, "totalPages": 0 } }	Requires admin or technician. 401, 403.
Print activity	GET	/api/dashboard/print-activity	Returns week or month print activity chart data.	Query: range=week or range=month.	{ "data": { "range": "week", "points": [] } }	Requires admin or technician. 401, 403.
Logs overview	GET	/api/logs/overview	Returns operational log overview metrics.	Query: range=day, week, or month.	{ "data": { "printJobs": 0, "failedJobs": 0, "heldJobs": 0 } }	Requires admin or technician. 401, 403, 422.
Operational events	GET	/api/logs/operational-events	Lists operational events from jobs, alerts, and audit records.	Query: range, search, type, page, limit.	{ "data": { "data": [], "total": 0, "page": 1, "limit": 25 } }	Requires admin or technician. 401, 403, 422.
List alerts	GET	/api/alerts	Lists device and operational alerts.	Query: search, status=active acknowledged all, severity=critical warning info.	{ "data": [{ "id": "alert-id", "title": "AD sync delay", "status": "active" }] }	Requires admin or technician. 401, 403, 422.
Get alert	GET	/api/alerts/:id	Returns a single alert.	Path: id.	{ "data": { "id": "alert-id", "title": "Printer offline" } }	Requires admin or technician. 401, 403, 404.
Acknowledge alert	POST	/api/alerts/:id/acknowledge	Marks an alert as acknowledged by the current user.	Path: id. No body.	{ "data": { "id": "alert-id", "status": "acknowledged" } }	Requires admin or technician. 401, 403, 404.

4.6 User, Group, Printer, and Queue Endpoints

Endpoint name	Method	Route	Description	Request parameters and body example	Response example	Errors and auth
List users	GET	/api/users	Lists users with search, status, role, and group filters.	Query: search, status, role, groupName, page, limit.	{ "data": [{ "id": "uuid", "email": "student@university.edu" }], "total": 1 }	Requires admin or technician. 401, 403, 422.
Create user	POST	/api/users	Creates a user, credential, role, quota, optional group membership, and audit log.	{ "username": "new.user", "email": "new@university.edu", "displayName": "New User", "password": "123456", "role": "standard_user" }	{ "data": { "id": "uuid", "username": "new.user" } }	Requires admin or technician. Technicians may create standard users only. 409 duplicate. 422 invalid body.
List user groups	GET	/api/users/groups	Returns group names for user forms and filters.	No parameters.	{ "data": ["CCM-Students", "Technicians"] }	Requires admin or technician. 401, 403.
Get user	GET	/api/users/:id	Returns a user by UUID or internal ID. The special id me resolves to the current user.	Path: id.	{ "data": { "id": "uuid", "display_name": "Student User" } }	Requires Bearer token. 404 if missing.
Update user	PATCH	/api/users/:id	Updates user profile, role, group, suspension flag, or quota fields.	{ "displayName": "Updated User", "allocatedPages": 1000 }	{ "data": { "id": "uuid", "display_name": "Updated User" } }	Requires admin or technician. Technician role restrictions apply. 403, 404, 422.
Suspend user	POST	/api/users/:id/suspend	Suspends a user account.	Path: id.	{ "data": { "message": "User suspended" } }	Requires admin or technician. Administrator accounts are view-only. 403, 404.
Reactivate user	POST	/api/users/:id/reactivate	Reactivates a suspended user account.	Path: id.	{ "data": { "message": "User reactivated" } }	Requires admin or technician. 403, 404.
Update quota	PATCH	/api/users/:id/quota	Updates allocated pages for a user.	{ "allocatedPages": 1200 }	{ "data": { "id": "uuid", "quota_total": 1200 } }	Requires admin or technician. 403, 404, 422.
Delete user	DELETE	/api/users/:id	Deletes a user and related dependent data after recording an audit log.	Path: id.	204 No Content.	Requires admin or technician. Administrator accounts are view-only. 403, 404.
List groups	GET	/api/groups	Lists AD-style groups with member and quota metadata.	Query: search, page, limit.	{ "data": [{ "id": "group-uuid", "name": "CCM-Students" }], "total": 1 }	Requires admin or technician. 401, 403, 422.
Create group	POST	/api/groups	Creates an AD-style group record.	{ "name": "AI Lab", "quotaPeriod": "Monthly", "initialBalance": 1500, "initialRestriction": false, "defaultForNew Users": false }	{ "data": { "id": "group-uuid", "name": "AI Lab" } }	Requires admin. 409 duplicate. 422 invalid body.
List group users	GET	/api/groups/:id/users	Lists users assigned to a group.	Path: id. Query: search, page, limit.	{ "data": [{ "id": "user-uuid", "groups": ["CCM-Students"] }], "total": 1 }	Requires admin or technician. 404, 422.
Get group	GET	/api/groups/:id	Returns group details.	Path: id.	{ "data": { "id": "group-uuid", "name": "CCM-Students" } }	Requires admin or technician. 404.
Update group	PATCH	/api/groups/:id	Updates group details and quota policy metadata.	{ "name": "CCM-Students", "quotaPeriod": "Monthly", "initialBalance": 1000, "initialRestriction": true, "defaultForNew Users": true }	{ "data": { "id": "group-uuid", "initial_balance": 1000 } }	Requires admin. 404, 409, 422.
Delete group	DELETE	/api/groups/:id	Deletes a group after updating dependent queue access rules.	Path: id.	204 No Content.	Requires admin. 404, 409.

Endpoint name	Method	Route	Description	Request parameters and body example	Response example	Errors and auth
List printers	GET	/api/printers	Lists printers with status, search, and pagination filters.	Query: status, search, page, limit.	{ "data": [{ "id": "printer-uuid", "name": "HP MFP M830 - 22/339" }], "total": 1 }	Requires admin or technician. 401, 403, 422.
Create printer	POST	/api/printers	Creates a printer record.	{ "name": "Printer A", "status": "online", "connectorType": "windows_queue", "connectorTarget": "QueueName" }	{ "data": { "id": "printer-uuid", "name": "Printer A" } }	Requires admin. 409 duplicate. 422 invalid body.
Get printer	GET	/api/printers/:id	Returns a printer record.	Path: id.	{ "data": { "id": "printer-uuid", "connector_type": "hp_pjl_stored_job" } }	Requires admin. 404.
Update printer	PATCH	/api/printers/:id	Updates printer metadata, status, connector type, target, and capabilities.	{ "status": "maintenance", "tonerLevel": 42 }	{ "data": { "id": "printer-uuid", "status": "maintenance" } }	Requires admin. 404, 422.
Delete printer	DELETE	/api/printers/:id	Archives a printer by setting status to archived.	Path: id.	204 No Content.	Requires admin. 404.
Printer errors	GET	/api/printers/:id/errors	Lists device_errors for a printer.	Path: id.	{ "data": [{ "title": "Printer offline", "status": "open" }] }	Requires admin. 404.
List queues	GET	/api/queues	Lists non-archived queues.	Query: status, search, page, limit.	{ "data": [{ "id": "queue-uuid", "name": "Student Default Queue" }], "total": 1 }	Requires Bearer token. 422 invalid filters.
My eligible queues	GET	/api/queues/eligible/mine	Lists queues available to the current user.	No parameters.	{ "data": [{ "id": "queue-uuid", "release_mode": "secure_release" }] }	Requires Bearer token.
Create queue	POST	/api/queues	Creates a queue, assigns printers, access rules, pricing, and audit data.	{ "name": "Lab Queue", "queueType": "student", "releaseMode": "secure_release", "printerIds": [], "allowedGroups": ["CCM-Students"], "costPerPage": 1 }	{ "data": { "id": "queue-uuid", "name": "Lab Queue" } }	Requires admin. 409 duplicate. 422 invalid body.
Get queue	GET	/api/queues/:id	Returns queue details, assigned printers, and recent queue logs.	Path: id.	{ "data": { "id": "queue-uuid", "printers": [], "queue_logs": [] } }	Requires Bearer token. 404.
Update queue	PATCH	/api/queues/:id	Updates queue metadata, printers, access rules, and pricing.	{ "status": "active", "retentionHours": 24, "allowedGroups": ["CCM-Students"] }	{ "data": { "id": "queue-uuid", "status": "active" } }	Requires admin. 404, 422.
Delete queue	DELETE	/api/queues/:id	Archives a queue when it has no pending jobs.	Path: id.	204 No Content.	Requires admin. 409 if pending jobs exist. 404.

4.7 Development and Connector Endpoints

Endpoint name	Method	Route	Description	Request parameters and body example	Response example	Errors and auth
Direct print diagnostic	POST	/dev/print-direct	Diagnostic path that converts PDF to PostScript and sends to raw socket printer target.	multipart/form-data: file=document.pdf.	{ "jobId": "uuid", "status": "sent_to_printer" }	No auth middleware in current code. Should only be exposed on trusted networks. 400 missing file. 500 connector/conversion failure.
Windows queue diagnostic	POST	/dev/print-windows-queue	Diagnostic path that forwards a PDF to the Windows connector queue print endpoint.	multipart/form-data: file=document.pdf, optional printerName.	{ "status": "queued", "printerName": "HP-MB30-22-339" }	No auth middleware in current code. 400 missing file. 500 connector fetch failure.
Connector health	GET	Connector /health	Checks connector process, queue target, queue share, and print mode.	No parameters.	{ "ok": true, "printerName": "HP-MB30-22-339", "printMode": "sumatra" }	Public in current connector code.
Connector queue print	POST	Connector /print	Submits uploaded PDF to Windows print command path.	multipart/form-data: file=document.pdf, optional jobId, optional printerName.	{ "jobId": "uuid", "status": "queued", "command": "...", "queuedAt": "timestamp" }	Requires Bearer token only when WINDOWS_PRINT_CONNECTOR_TOKEN is configured. 401 invalid connector token. 500 command failure.
Connector store on device	POST	Connector /store-on-device	Stores uploaded PDF in HP device memory using username, jobName, and PIN metadata.	multipart/form-data: file, username, jobName, pin, printerHost, printerPort, copyCount.	{ "jobId": "uuid", "status": "stored_on_device", "username": "PRINTSOL_TEST", "bytesSent": 12345 }	Requires connector token when configured. 400 missing file. 500 printer or socket failure.
Connector PJI diagnostic	POST	Connector /diagnostics/pji-stored-job	Sends a test HP PJI stored/private job to validate printer-panel retrieval.	JSON body may include printerHost and printerPort. Empty body is accepted.	{ "jobId": "PJI_STORE_001", "status": "stored_on_device", "pin": "1234" }	Requires connector token when configured. 500 printer or socket failure.
Connector raw PDF print	POST	Connector /print-file	Accepts a raw PDF request body and submits it to the Windows queue command path.	Raw application/pdf body. Query: jobId, printerName. Header: x-file-name.	{ "jobId": "uuid", "status": "queued", "uploadedPath": "..." }	Requires connector token when configured. 400 empty body. 500 command failure.

5. Deployment and Infrastructure

5.1 Local Setup Instructions

1. Install Node.js 20 or later and npm 10 or later.
2. Install Docker Desktop or provide another PostgreSQL instance.
3. Install backend dependencies from backend with npm install.
4. Install frontend dependencies from frontend with npm install.
5. Create backend/.env from backend/.env.example or configure equivalent environment variables.
6. Start PostgreSQL or point DATABASE_URL at an existing instance.
7. Run backend migrations with npm run db:migrate from backend.
8. Start the backend with npm run dev from backend.
9. Start the frontend with npm run dev from frontend.

5.2 Environment Variables

Variable	Purpose
PORT	Main backend HTTP port. Default 4000.
FRONTEND_ORIGIN	Comma-separated list of allowed CORS origins for frontend development or deployment.
DATABASE_URL	PostgreSQL connection string.
PRINTER_HOST	Fallback raw socket printer host.
PRINTER_PORT	Fallback raw socket printer port. Default 9100.
WINDOWS_PRINT_QUEUE_TARGET	Windows queue name used by connector paths.
WINDOWS_PRINT_QUEUE_SHARE	Recorded shared queue path for metadata and operator context.
WINDOWS_PRINT_CONNECTOR_URL	Main backend URL for the Windows connector. Used by WindowsQueuePrintClient and HpPjlStoredJobClient.
WINDOWS_PRINT_CONNECTOR_TOKEN	Optional Bearer token shared between main backend and connector.
WINDOWS_CONNECTOR_PORT	Connector HTTP port. Default 4100.
WINDOWS_CONNECTOR_UPLOAD_DIR	Connector-side upload directory for PDFs received by connector endpoints.
WINDOWS_PRINT_MODE	Connector print strategy: sumatra, powershell, or command.
SUMATRA_PDF_PATH	Path to SumatraPDF executable for Windows queue submission.
WINDOWS_PRINT_COMMAND	Custom command template when WINDOWS_PRINT_MODE is command.
WINDOWS_PRINT_TIMEOUT_MS	Print command timeout in milliseconds.
JWT_SECRET	HMAC secret used to sign current JWT sessions.
JOB_PIN_ENCRYPTION_KEY	Secret used to encrypt stored-job PINs in the database.
GHOSTSCRIPT_BIN	Ghostscript executable path for PDF to PostScript conversion diagnostics.
UPLOAD_DIR	Main backend PDF upload directory.
CONVERTED_DIR	Directory for converted PostScript files.
MIGRATIONS_DIR	SQL migrations directory.
VTE_BACKEND_URL	Frontend build-time backend URL. Defaults to http://localhost:4000 when omitted.

5.3 Build and Run Steps

5.3.1 Backend

```
cd backend
npm install
npm run db:migrate
```



```
npm run dev
```

Production-style build and start scripts are available:

```
cd backend
npm run build
npm start
```

5.3.2 Frontend

```
cd frontend
npm install
npm run dev
```

Production-style build and preview scripts are available:

```
cd frontend
npm run build
npm run preview
```

5.3.3 Windows Connector

```
cd backend
npm run dev:windows-connector
```

After starting the connector, verify:

```
Invoke-WebRequest http://localhost:4100/health
```

5.4 Deployment on College VM

1. Provision Node.js and npm on the VM that will run the main backend.
2. Provision PostgreSQL locally, through Docker, or as a reachable database service.
3. Set backend .env values for DATABASE_URL, FRONTEND_ORIGIN, JWT_SECRET, JOB_PIN_ENCRYPTION_KEY, and printer connector values.
4. Run npm install and npm run build in backend.
5. Run npm run db:migrate against the target database.
6. Start the main backend using npm start after build, or npm run dev during pilot testing.
7. Build the frontend with VITE_BACKEND_URL pointing at the backend host.
8. On the Windows-side connector machine, install dependencies, configure WINDOWS_CONNECTOR_PORT, WINDOWS_PRINT_QUEUE_TARGET, WINDOWS_PRINT_QUEUE_SHARE, WINDOWS_PRINT_MODE, SUMATRA_PDF_PATH, and connector token if used.
9. Open required firewall access between the main backend and connector on port 4100, between clients and backend on port 4000, and from connector to the printer where required.
0. Verify backend /health, connector /health, login, job submission, store-on-device, and printer-panel retrieval.

5.5 Required Services and Ports

Service	Default port	Required for
PostgreSQL	5432	All DB-backed application behavior.
Main backend	4000	Frontend API traffic and /dev diagnostic routes.
Frontend dev server	5173	Local developer UI.
Windows connector	4100	Store-on-device and Windows queue print submission.
Printer raw socket	9100	HP PjL stored/private job and raw socket diagnostic paths.

5.6 Logging and Monitoring

Source	What it records
Process stdout and stderr	Server startup, direct printer target, connector startup, diagnostic connector output, and temporary device PIN debug logs currently emitted by jobs.service.ts.
auth_logs	Successful and failed sign-in attempts.
audit_logs	User, queue, printer, and group management actions with before and after state where implemented.
print_job_events	Job lifecycle events such as submitted, device_storage_prepared, submitted_to_device_storage, released, failed, and cancelled.
print_logs	Operational job rows used by dashboards and log views.
device_errors	Printer and queue errors used by alert screens.

5.7 Troubleshooting Deployment Issues

Symptom	Likely cause	Resolution
npm run db:migrate fails with ECONNREFUSED on port 5432.	PostgreSQL is not running or DATABASE_URL points at the wrong host, port, or credentials.	Start Docker Desktop and docker compose up -d postgres from backend, or correct DATABASE_URL.
Frontend shows NetworkError or CORS failure.	FRONTEND_ORIGIN does not include the exact browser origin.	Set FRONTEND_ORIGIN to include every local or deployed frontend origin, comma-separated.
Store on printer returns fetch failed.	Main backend cannot reach WINDOWS_PRINT_CONNECTOR_URL.	Start npm run dev:windows-connector on the connector machine, verify /health, check firewall and IP/port.
Job status is failed, must be held to release.	The previous release attempt failed and the backend marked the job failed.	Submit a new job after resolving connector availability. Current service only releases held jobs.
Connector accepts job but no final completion is visible.	Completion telemetry is not implemented.	Verify printer queue or printer panel manually. Treat connector acceptance as submission, not final paper output.
Color, duplex, or paper options are not visible in portal.	The current connector path does not enforce these options reliably.	Use queue defaults until PrintTicket, separate queues, HP PJP/PCL/PostScript, or another reliable enforcement path is implemented.

6. Maintenance and Extensibility Guide

6.1 How to Add New Features

1. Identify the feature area under frontend/src/features or create a new feature directory if the domain is new.
2. Add or update backend service logic before adding route behavior when the feature changes application data.
3. Add SQL migrations for persistent schema changes. Do not modify old migrations that have already been applied.
4. Add route validation with zod for any new request body or query string.
5. Add frontend API adapter functions for the new backend route.
6. Render state through existing table, filter, dialog, badge, page header, and layout components when possible.
7. Run backend typecheck/build and frontend build/lint before deployment.

6.2 How to Modify the Print Workflow

Print workflow behavior is centered in jobs.service.ts and print-delivery-service.ts. Keep database state changes explicit and event-backed. When adding a delivery channel, add the connector type to the printer connector_type constraint through a migration, implement a connector client or service, and update PrintDeliveryService to route by connector_type.

Job lifecycle changes should also update print_jobs status constraints, print_logs constraints, frontend status mapping in portal submit-job/api.ts, and any dashboard or logs queries that count status groups.

6.3 How to Extend Portals

Portal	Extension pattern
Portal	Add screens under frontend/src/features/portal, route them in app/router.tsx under /portal, and use portal/session or portal API adapters for user context.
Admin	Add screens under frontend/src/features/admin, add navigation to AdminShell if needed, and secure backend routes with requireRole("admin").
Technician	Add screens under frontend/src/features/technician, route under /tech, and enforce backend permissions with requireRole("admin", "technician") plus service-level restrictions where necessary.

6.4 How to Add New APIs

1. Create or extend a router under backend/src/routes.
2. Add authenticate middleware if the endpoint requires a user.
3. Add requireRole for privileged operations.
4. Define zod schemas and call validateBody or validateQuery.
5. Implement data operations in a service module, using transactions for multi-table changes.
6. Return data through ok, created, accepted, noContent, or paginated helpers.
7. Mount the router from backend/src/routes/index.ts if it is a new route group.
8. Add a frontend API adapter in the relevant feature directory.

6.5 How to Maintain Code Quality

- Keep business logic in backend services, not route handlers.
- Keep frontend HTTP details in feature API adapters and the shared API client, not directly in UI controls.
- Use zod for request validation and database constraints for persistent integrity.
- Use explicit SQL migrations and treat backend/migrations as the schema source of truth.
- Use audit_logs for administrative changes and print_job_events for job lifecycle changes.
- Avoid exposing diagnostic /dev routes or connector endpoints without network protection.
- Prefer existing UI components and layout conventions over new one-off components.

6.6 Coding Conventions Used

Area	Convention
Backend modules	ES modules with TypeScript, .js import extensions in source imports, async service functions, direct SQL queries.
Validation	zod schemas in route files, request parsing middleware, multer upload filters for PDFs.
Database	Plain SQL migrations, pg pool, transaction helper, UUID public IDs with numeric internal primary keys.
Frontend modules	Feature directories, typed API adapters, shared UI components, React Router nested layouts.
Styling	Tailwind CSS utility classes, shared UI primitives, lucide-react icons.

7. Testing and Validation

7.1 Testing Strategy

The repository currently emphasizes build validation, type checking, linting, database migrations, and manual workflow validation. No dedicated automated test suite is present in the package scripts. Future work should add automated backend service tests, route tests, frontend component tests, and connector integration tests.

7.2 Validation Commands

Area	Command	Purpose
Backend build	cd backend; npm run build	Compiles TypeScript backend to dist.
Backend typecheck	cd backend; npm run typecheck	Runs TypeScript compiler without emitting files.
Database migrations	cd backend; npm run db:migrate	Applies pending SQL migrations.
Frontend build	cd frontend; npm run build	Type-checks and builds the Vite frontend.
Frontend lint	cd frontend; npm run lint	Runs ESLint across frontend files.

7.3 Manual Testing Steps

1. Start PostgreSQL and run migrations.
2. Start the backend and confirm GET /health returns { "ok": true }.
3. Start the frontend and open the configured frontend origin.
4. Sign in with seeded development credentials.
5. Verify admin dashboard, users, groups, printers, queues, and logs pages load from API-backed data.
6. Verify technician dashboard, printer list, user list, and alerts pages load.
7. Sign in as a standard user and submit a valid PDF.
8. Confirm the job appears in history as Ready to send.
9. Start the Windows connector and confirm connector /health responds.
0. Select Store on printer and confirm the job changes to Stored on printer with folder, job name, and PIN instructions.
1. Use the printer panel to retrieve from device memory using the displayed instructions.
2. Test cancellation on a newly held job before release.
3. Review print_job_events, print_logs, auth_logs, and audit_logs after workflows.

7.4 Key Test Scenarios

Scenario	Expected result
Invalid login	Backend returns 401 and logs failed auth attempt when logging does not fail.
Suspended user login	Backend returns 401 with account not allowed message.
PDF upload with valid quota	Job created with held status and file metadata stored.
Non-PDF upload	Upload rejected by multer file filter.
Quota exceeded	Job submission rejected with validation error.
Store held job on device	Backend prepares PIN metadata, connector accepts job, status becomes stored_on_device.
Store job when connector is unavailable	Backend records failure_reason and sets job status to failed.
Cancel held job	Status becomes cancelled and print_logs updates.
Cancel failed or stored job	Backend returns validation error because only held jobs can be cancelled.
Technician attempts privileged admin-only changes	Backend rejects with forbidden error where role checks or service restrictions apply.

7.5 Validation Checklist Before Deployment

- Backend .env is configured with production-safe JWT_SECRET and JOB_PIN_ENCRYPTION_KEY.
- DATABASE_URL points to the intended PostgreSQL instance.
- All migrations have run successfully.
- FRONTEND_ORIGIN matches deployed frontend origin exactly.
- VITE_BACKEND_URL points at the deployed backend.
- Connector /health is reachable from the main backend host.
- WINDOWS_PRINT_CONNECTOR_TOKEN is configured for non-local connector exposure.
- Firewall rules allow required traffic and block diagnostic routes from untrusted networks.
- Store-on-device workflow has been tested with a newly uploaded job.
- Printer-panel retrieval has been verified manually.
- Logs and audit tables are reviewed after test actions.

8. Active Directory Integration Plan

8.1 Where It Fits in the System

Active Directory integration is in progress. It should replace the current database-password login path while keeping PostgreSQL as the application authority for roles, suspensions, quotas, queue access, audit logs, and print history. The current schema is already prepared for AD object IDs and group membership through `users.ad_object_id`, `ad_groups`, and `user_ad_group_memberships`.

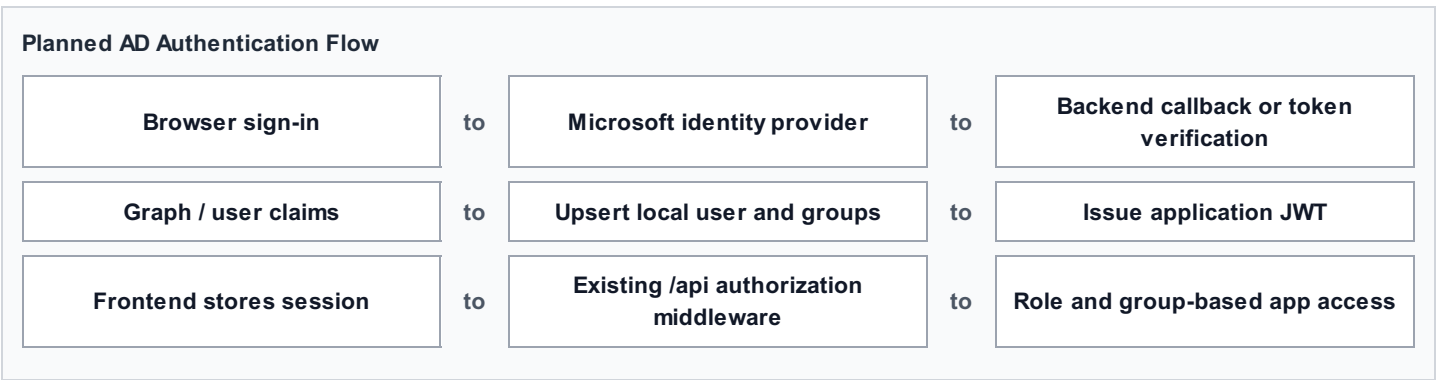
8.2 Current Repository Evidence

Evidence	Status	Meaning
<code>users.ad_object_id</code>	Implemented schema field	Can link a local user record to an external AD object.
<code>ad_groups</code> and <code>user_ad_group_memberships</code>	Implemented schema tables	Support group-based queue access and future identity sync.
<code>queue_access_rules</code> <code>rule_type</code> <code>ad_group</code>	Implemented access model	Queues can be made available to users based on group names.
SSO Test 1.py	In Progress proof of concept	Shows a Flask and MSAL Microsoft sign-in flow and Graph /me lookup. It is not integrated into the Express backend.
Current <code>/api/auth/login</code>	Implemented temporary login	Uses database credentials and JWTs until AD is integrated.

8.3 Required Changes

1. Add AD or Microsoft identity provider configuration to backend environment variables.
2. Replace or extend `/api/auth/login` with an SSO callback or token exchange flow suitable for the chosen identity provider.
3. Validate external identity tokens server-side and extract stable object ID, email, display name, and group identifiers.
4. Upsert users by `ad_object_id` or verified email.
5. Synchronize `user_ad_group_memberships` from AD or Graph group data.
6. Preserve local application roles, suspensions, quotas, technician privileges, and audit history in PostgreSQL.
7. Issue the same internal JWT shape after external identity verification so existing API authorization continues to work.
8. Remove or restrict development password credentials in production.

8.4 Integration Approach



8.5 Security Considerations

- Do not store AD client secrets in the repository.
- Validate identity tokens on the backend, not only in the browser.
- Restrict accepted tenants and domains according to college policy.
- Keep local suspension and role checks authoritative even when AD says the account is valid.
- Record successful and failed SSO attempts in `auth_logs`.
- Review `localStorage` token storage before production because browser token storage affects XSS risk.

- Define a controlled fallback path for administrators if external identity is unavailable.

9. Onboarding Guide

9.1 Steps for a New Developer

1. Read the root README and backend README.
2. Install Node.js, npm, and Docker Desktop or configure another PostgreSQL instance.
3. Install backend dependencies, create backend/.env, and run migrations.
4. Install frontend dependencies and start the Vite dev server.
5. Sign in with the seeded development accounts listed in the root README.
6. Walk through the portal, admin, and technician routes.
7. Review backend/src/routes to understand the API surface.
8. Review backend/src/services/jobs.service.ts before changing print behavior.
9. Review backend/migrations before changing database behavior.
0. Run build and validation commands before submitting changes.

9.2 Required Tools

Tool	Required for
Node.js 20 or later	Backend and frontend development.
npm 10 or later	Dependency installation and scripts.
Docker Desktop or PostgreSQL	Local database.
Ghostscript	Raw socket PDF to PostScript diagnostic path.
SumatraPDF on Windows connector machine	Windows queue print mode when WINDOWS_PRINT_MODE is sumatra.
PowerShell	Windows setup, connector checks, and VM operations.

9.3 Setup Checklist

- backend/node_modules installed.
- frontend/node_modules installed.
- backend/.env exists and points at a working database.
- npm run db:migrate reports database up to date.
- GET http://localhost:4000/health returns { "ok": true }.
- Frontend loads at the Vite URL.
- Login works with development credentials.
- Connector /health responds before testing Store on printer.

9.4 How to Run the System

<pre>cd backend npm install npm run db:migrate npm run dev</pre>
<pre>cd frontend npm install npm run dev</pre>
<pre>cd backend npm run dev:windows-connector</pre>

9.5 Common Pitfalls

Pitfall	Explanation
Using localhost and 127.0.0.1 interchangeably	The browser treats them as different origins. FRONTEND_ORIGIN must include the exact origin used.
Running only the backend when testing secure release	Store on printer requires the Windows connector at WINDOWS_PRINT_CONNECTOR_URL.
Retrying a failed release job	The backend only releases jobs in held status. A connector failure marks the job failed, so create a new job after fixing the connector.
Expecting final print completion telemetry	Current implementation records submission and stored-job state, not final printed-page confirmation.
Changing schema without migration	The backend uses plain SQL migrations. Schema changes must be recorded in backend/migrations.

10. Future Improvements

10.1 Planned Enhancements

Enhancement	Status	Rationale
Active Directory login and identity sync	In Progress	Replace development DB credentials with institutional authentication while preserving local application authorization.
Production cleanup for expired jobs and stored files	Planned	Use expires_at and deleted_at fields to remove held files and stale metadata safely.
Reliable physical completion telemetry	Planned	Detect final printed state, device faults, and actual completion rather than connector acceptance only.
Connector authentication and firewall hardening	Planned	Ensure connector endpoints are not exposed without a token and network restrictions.
Automated tests	Planned	Add route, service, migration, and frontend tests to reduce regression risk.
Print option enforcement	Planned	Add reliable enforcement for color, duplex, and paper options through queue defaults, separate queues, HP languages, PrintTicket, or another validated path.

10.2 Performance Improvements

- Add pagination and filtering consistently to high-volume operational views.
- Review dashboard aggregation queries under realistic production data volume.
- Add background workers for retention cleanup and delayed device polling if completion telemetry is added.
- Consider external file storage if local filesystem storage is not appropriate for deployment scale.

10.3 Security Upgrades

- Complete Active Directory integration and remove development credentials from production.
- Move production secrets to VM or platform secret management.
- Require connector tokens outside local development.
- Restrict or remove unauthenticated /dev routes before production exposure.
- Review browser token storage and consider safer session handling for production.
- Remove or gate temporary debug logging that prints generated device PINs.

Final implementation summary: The repository implements a full-stack MVP with secure print release through an HP P.J.L stored/private job path. Active Directory integration is in progress, and production readiness depends on identity integration, connector hardening, retention cleanup, and physical printer completion telemetry.